How I Built a SaaS in 7 Days Using AI Agents — The Timeo Story

> *A war story for solo founders, indie hackers, and developers who are tired of "move fast and break things" being just a slogan.*

By Jabez | Oxloz LLC Version 1.0 — March 2026

Price: $29 | Not for redistribution

A Note Before We Begin

I'm going to tell you something that sounds like a lie: I built a production-ready, multi-tenant SaaS — with NFC door controllers, facial recognition, data migration for 381 users, mobile apps on both iOS and Android, Malaysian e-invoicing compliance, and a QA-hardened codebase — in 7 days.

I didn't work 24 hours a day. I slept. I ate. I had conversations with my team.

What I had was a workflow. A set of AI agents. And a very stubborn belief that if you set up the right scaffolding, the work almost builds itself.

This isn't a motivational book. I'm not going to tell you to "believe in yourself" or "embrace the hustle." I'm going to show you exactly what I did, why I made the decisions I made, and give you the templates, prompts, and checklists I used so you can do it too.

The product is called **Timeo** — a gym management SaaS built for the Malaysian market. It's real. It's live. It's making money.

Let's get into it.

Chapter 1: The Idea — Why Gym Management? Why Malaysia?

The Market Gap Nobody Talks About

If you're building SaaS in 2026, the first thing most people tell you is "don't build for a niche, build for scale." They're wrong.

Niche is where the money hides.

Here's the thing about gym management software in Malaysia: the big players — Mindbody, Glofox, Gymmaster — were all built for Western markets. Their pricing is in USD. Their support is in a timezone that's 8–12 hours behind Malaysia. Their UX assumes a certain literacy with software that doesn't match a typical independent gym owner in Johor Bahru or Penang.

I found this gap the boring way: I talked to gym owners. Ten of them in one week. I asked them what software they used. Eight said they were using WhatsApp groups and Excel. One was using a Filipino-built app that kept going down. One was using a Malaysian competitor that hadn't shipped a new feature in two years.

That last one was the signal. A stagnant local competitor with paying customers is a gift. It means the market exists, the willingness to pay exists, but the product is underserving them.

The Competitor Research (How I Did It With AI)

I didn't spend weeks on this. I spent three hours with Claude.

Here's the prompt I used:

```
You are a market research analyst specializing in B2B SaaS in Southeast Asia.

I'm building gym management software for the Malaysian market.

Key context:
- Malaysia is a predominantly Muslim country — prayer time integration matters
- Mix of English and Bahasa Malaysia in UI
- Local payment methods: FPX, DuitNow, Touch 'n Go eWallet
```

```
- Hardware: some gyms use biometric terminals, others use QR-based check-in
- Market size: ~3,000 independent gyms in Malaysia (not chains)

Research the following competitors and give me:
1. Their pricing model
2. Their known pain points (from app store reviews, Reddit, forums)
3. Feature gaps I could exploit

Competitors to research:
- Mindbody
- Glofox
- [LOCAL_COMPETITOR_NAME]
- GymMaster

Also suggest 5 features that would be a "only in Malaysia" differentiator.
```

The output gave me a positioning framework in 20 minutes that would have taken a consultant a week. The five "only in Malaysia" features that came out of that session became our core differentiators:

1. **FPX and DuitNow payment integration** — not just Stripe
2. **Thermal receipt printing** in the format Malaysian tax authorities expect
3. **Prayer time-aware scheduling** — gym classes don't conflict with Subuh or Maghrib
4. **Bahasa Malaysia / English bilingual UI** — toggle per user, not per tenant
5. **NFC + facial recognition turnstile integration** for the hardware-forward gyms

That last one turned out to be the hardest to build. We'll get to Day 4.

What I Decided NOT to Build

This is as important as what you build.

I made a deliberate list of things we would not do in v1:

- No custom workout programming (leave that to Trainerize)
- No nutrition tracking
- No marketplace for personal trainers
- No group class livestreaming
- No complex reporting dashboards with 47 chart types

The product needed to do five things extremely well: 1. Manage members and their subscriptions 2. Handle check-ins (QR and hardware) 3. Process payments and generate compliant receipts 4. Let gym staff manage roles and access 5. Work on mobile (native iOS and Android)

Everything else was "Phase 2" — which is code for "probably never, and that's fine."

The Business Model

Before writing a single line of code, I validated the pricing with three gym owners.

```
 Tier 1 (Starter): RM 99/month — up to 100 members, 1 branch
Tier 2 (Growth):  RM 199/month — up to 500 members, 3 branches
Tier 3 (Scale):   RM 399/month — unlimited members, unlimited branches
Hardware bundle:  RM 1,500 one-time for NFC controller + tablet setup
```

Two of the three said yes on the spot. The third said "maybe when we grow." That's a yes in disguise.

With a target of 50 paying gyms in 6 months (conservative), that's RM 9,950–RM 19,950/month in ARR at month 6. The business was real before I wrote a function.

Chapter 2: Day 1 — Architecture (Getting It Right Before Getting It Done)

Why Architecture Day Matters

Most solo founders skip architecture day. They open VS Code, install a Next.js template, and start building whatever feels exciting. Two weeks later they hit a wall — multi-tenancy doesn't work, their schema is a mess, and they're doing full rewrites.

I've done that. I refuse to do it again.

Day 1 was entirely planning and scaffolding. No features. No UI. Just making sure that whatever we built from Day 2 onward would stand up.

The Stack Decision

I'm going to be opinionated here: the stack doesn't matter as much as people think, but it does matter that you have strong opinions about it and stick to them.

For Timeo, I chose:

Frontend: Next.js 15 (App Router) with TypeScript **Backend:** Hono (deployed on Cloudflare Workers) **Database:** PostgreSQL on Supabase **Auth:** Supabase Auth **File Storage:** Cloudflare R2 **Email:** Resend **Payments:** Stripe + local FPX via Billplz **Mobile:** Flutter (shared codebase for iOS + Android) **Deployment:** Vercel (frontend) + Cloudflare Workers (API)

Why Hono over Next.js API routes? Because Hono runs on the edge, is stupidly fast, has excellent TypeScript support, and its routing is explicit and readable. When you're moving fast, readable > clever.

Why Supabase over Neon or PlanetScale? Row-level security. For multi-tenant SaaS, Postgres RLS is the cleanest way to enforce tenant isolation at the database layer without polluting every query with `WHERE tenant_id = ?`. Supabase makes RLS manageable.

The Multi-Tenancy Decision

This is the architectural decision that breaks most gym management software competitors. Get it wrong and you either:

a) Leak data between tenants (catastrophic) b) Make it impossible to query across tenants for analytics (annoying) c) End up with a schema so complex nobody can maintain it

I used the **shared schema, row-level security** approach. One database. All tenants in the same tables. RLS policies ensure each tenant only sees their data.

Here's the core schema design I generated with AI:

```
 -- Organizations (gyms) are the top-level tenant
CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
```

```
  plan TEXT NOT NULL DEFAULT 'starter',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Every entity has org_id
CREATE TABLE members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  full_name TEXT NOT NULL,
  phone TEXT,
  email TEXT,
  ic_number TEXT, -- Malaysian IC number
  photo_url TEXT,
  status TEXT DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- RLS: members only visible within same org
ALTER TABLE members ENABLE ROW LEVEL SECURITY;

CREATE POLICY "members_org_isolation" ON members
  USING (org_id = current_setting('app.current_org_id')::UUID);

-- Branches (multi-location support)
CREATE TABLE branches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES organizations(id),
  name TEXT NOT NULL,
  address TEXT,
  timezone TEXT DEFAULT 'Asia/Kuala_Lumpur'
);
```

The key insight: `current_setting('app.current_org_id')` is set at the start of every database session by the API layer. The API reads the tenant from the JWT, sets this session variable, and every query is automatically scoped. No `WHERE org_id = ?` in application code.

The Prompt I Used to Generate the Schema

I used Claude Opus for architecture work. Sonnet for implementation. More on that philosophy in Chapter 9.

```
You are a senior database architect specializing in multi-tenant SaaS systems.

I'm building a gym management SaaS called Timeo for the Malaysian market.

Requirements:
- Multi-tenant (organizations = gyms)
- Multi-branch per organization
- Members with Malaysian IC numbers, photos, QR codes
- Subscriptions with Malaysian pricing (MYR)
- Check-in tracking (QR scan, NFC, facial recognition)
- Role-based access: Owner, Manager, Staff, Member
- Full audit trail for all mutations
- Soft deletes for members (GDPR-lite)

Design a complete PostgreSQL schema with:
1. All tables with proper indexes
2. RLS policies for tenant isolation
3. Enum types for status fields
4. Triggers for audit logging
5. Explain your decisions, especially around multi-tenancy

Output as executable SQL. Annotate complex parts.
```

The output was 400 lines of clean SQL. I reviewed it, made maybe 20 tweaks, and it became our production schema.

Monorepo Setup

```
timeo/
├── apps/
│   ├── web/          # Next.js dashboard
│   ├── api/          # Hono API
│   └── mobile/       # Flutter app
├── packages/
│   ├── types/        # Shared TypeScript types
│   ├── utils/        # Shared utilities
│   └── ui/           # Shared UI components (shadcn)
├── supabase/
│   ├── migrations/
```

```
│   └── seed.sql
└── docs/
    └── TIMEO_RULES.md  # (See Appendix)
```

I used `pnpm workspaces` for the JS/TS side and kept Flutter separate since it has its own package manager.

End of Day 1

By end of Day 1, I had: - ✅ Complete database schema with RLS - ✅ Monorepo scaffold - ✅ Supabase project set up with migrations - ✅ Next.js + Hono base apps running - ✅ Cloudflare Workers deployed (hello world) - ✅ Vercel project linked - ✅ Shared TypeScript types defined

No features. Just infrastructure. And it felt *great*.

Chapter 3: Day 2 — Core Features (The Hardest Day)

Why Day 2 Is the Hardest

Day 1 is fun — it's all architecture and ideas. Days 3-7 get more specific. Day 2 is where you face the full scope of what you've committed to building.

The core features for Timeo were: 1. Member registration and management 2. QR code generation and scanning 3. Check-in system 4. Role-based access control (RBAC) 5. Basic subscription management

This sounds like a week's work. With AI agents and a solid architecture foundation, it was one day.

The RBAC System

RBAC is one of those things that's conceptually simple but implementationally annoying. I used AI to generate both the database layer and the application middleware in one shot.

The roles:

```
 export const ROLES = {
  OWNER: 'owner',       // Full access, billing, settings
  MANAGER: 'manager',   // Manage members, staff, reports
  STAFF: 'staff',       // Check-ins, view members, basic ops
  MEMBER: 'member',     // Self-service portal only
} as const;

export const PERMISSIONS = {
  MEMBER_CREATE: 'member:create',
  MEMBER_UPDATE: 'member:update',
  MEMBER_DELETE: 'member:delete',
  MEMBER_VIEW: 'member:view',
  CHECKIN_MANAGE: 'checkin:manage',
  BILLING_MANAGE: 'billing:manage',
  STAFF_MANAGE: 'staff:manage',
  REPORT_VIEW: 'report:view',
  SETTINGS_MANAGE: 'settings:manage',
} as const;

export const ROLE_PERMISSIONS: Record<string, string[]> = {
  owner: Object.values(PERMISSIONS),
  manager: [
    PERMISSIONS.MEMBER_CREATE,
    PERMISSIONS.MEMBER_UPDATE,
    PERMISSIONS.MEMBER_VIEW,
    PERMISSIONS.CHECKIN_MANAGE,
    PERMISSIONS.STAFF_MANAGE,
    PERMISSIONS.REPORT_VIEW,
  ],
  staff: [
    PERMISSIONS.MEMBER_VIEW,
    PERMISSIONS.CHECKIN_MANAGE,
  ],
  member: [],
};
```

The Hono middleware for permission checking:

```
 // middleware/rbac.ts
export const requirePermission = (permission: string) => {
```

```
  return async (c: Context, next: Next) => {
    const user = c.get('user');
    const userRole = user.role;

    const allowed = ROLE_PERMISSIONS[userRole]?.includes(permission);

    if (!allowed) {
      return c.json({ error: 'Forbidden', code: 'INSUFFICIENT_PERMISSIONS' }, 403);
    }

    return next();
  };
};

// Usage in routes
app.post('/members',
  requirePermission(PERMISSIONS.MEMBER_CREATE),
  async (c) => {
    // handler
  }
);
```

QR Code System

Every member gets a unique QR code. The code encodes a signed token (not the member ID directly — that's a security smell).

```
 // Generate a QR token for a member
export async function generateMemberQR(memberId: string, orgId: string) {
  const payload = {
    sub: memberId,
    org: orgId,
    type: 'checkin',
    iat: Math.floor(Date.now() / 1000),
    // QR codes are long-lived but revocable via DB
    exp: Math.floor(Date.now() / 1000) + (365 * 24 * 60 * 60),
  };

  const token = await signJWT(payload, process.env.QR_SECRET!);
```

```
  // Store QR token hash in DB (for revocation)
  await db.memberQR.upsert({
    where: { member_id: memberId },
    create: { member_id: memberId, token_hash: hashToken(token), org_id: orgId },
    update: { token_hash: hashToken(token) },
  });

  return token;
}

// Scan a QR code during check-in
export async function processCheckin(token: string, branchId: string) {
  const payload = await verifyJWT(token, process.env.QR_SECRET!);

  // Verify token hasn't been revoked
  const qrRecord = await db.memberQR.findFirst({
    where: {
      member_id: payload.sub,
      token_hash: hashToken(token)
    }
  });

  if (!qrRecord) throw new Error('QR code revoked or invalid');

  // Check member is active and subscription is valid
  const member = await db.members.findFirst({
    where: { id: payload.sub, status: 'active' },
    include: { active_subscription: true }
  });

  if (!member?.active_subscription) {
    return { success: false, reason: 'subscription_expired' };
  }

  // Record check-in
  await db.checkins.create({
    data: {
      member_id: payload.sub,
      branch_id: branchId,
      org_id: payload.org,
      method: 'qr',
```

```
      checked_in_at: new Date(),
    }
  });

  return { success: true, member };
}
```

The AI Prompt Strategy for Features

Here's something I learned the hard way: don't ask AI to "build a feature." Ask it to build a *specific implementation* with *specific constraints*.

Bad prompt: "Build a member management system for my gym SaaS."

Good prompt:

```
 Context: I'm building a gym management SaaS (Timeo) using Hono + TypeScript +
PostgreSQL/Supabase. Multi-tenant with RLS. Strict TypeScript throughout.

Task: Implement the member CRUD API endpoints.

Schema (already exists):
[paste schema]

Shared types (already defined):
[paste types]

Requirements:
- POST /members — create member, generate QR token, send welcome SMS (Twilio)
- GET /members — paginated list with search (name, IC number, phone)
- GET /members/:id — single member with subscription status
- PATCH /members/:id — update member (partial updates only)
- DELETE /members/:id — soft delete (set status = 'deleted')

All endpoints require auth middleware (already set up).
POST/PATCH/DELETE require MEMBER_CREATE/UPDATE/DELETE permission.

Return consistent API shapes: { data, meta?, error? }
Include Zod validation schemas for all inputs.
Do NOT use any ORM — use raw Supabase client queries for performance.
```

```
Output: Complete, working TypeScript code. No TODOs. No placeholder comments.
```

That level of specificity makes the difference between code you can ship and code you spend two hours fixing.

The Day 2 Velocity

By the time I closed my laptop on Day 2: - ✅ Complete RBAC system (DB + API + middleware) - ✅ Member CRUD (create, read, update, soft-delete) - ✅ QR code generation and scanning - ✅ Check-in recording (QR method) - ✅ Basic subscription model - ✅ Member dashboard in Next.js (list, detail, add form)

Chapter 4: Day 3 — Payments & Receipts (The Malaysia-Specific Hell)

Why Payments Was Its Own Day

Payments are always harder than you think. In Malaysia, they're harder still because:

1. The dominant local payment method is **FPX** (Financial Process Exchange) — a direct bank transfer standard unique to Malaysia
2. **DuitNow** (instant transfer) is increasingly common
3. **Touch 'n Go eWallet** is expected for consumer-facing apps
4. The **Malaysian Digital Economy Corporation (MDEC)** has e-invoicing requirements that kicked in for businesses above certain thresholds in 2024
5. Thermal receipt printing in a specific format is expected by gym owners who need physical records

None of this is handled by Stripe out of the box.

The Payment Stack

I ended up with a layered approach: - **Stripe** for international cards and subscription management (they handle the recurring logic well) - **Billplz** for FPX (Malaysian bank transfers) — they have a clean API and decent documentation - **Custom webhook handlers** for both, unified into a single payment event system

```
 // Unified payment event types
export type PaymentEvent = {
  provider: 'stripe' | 'billplz';
  event_type: 'payment.success' | 'payment.failed' | 'subscription.renewed' | 'subscript
  amount: number;
  currency: 'MYR';
  member_id: string;
  org_id: string;
  subscription_id?: string;
  reference: string;
  metadata: Record<string, unknown>;
};

// Webhook handler (Hono route)
app.post('/webhooks/billplz', async (c) => {
  const signature = c.req.header('X-Signature');
  const body = await c.req.text();

  // Verify Billplz signature
  if (!verifyBillplzSignature(body, signature, process.env.BILLPLZ_X_SIGNATURE_KEY!)) {
    return c.json({ error: 'Invalid signature' }, 401);
  }

  const data = parseBillplzWebhook(body);

  if (data.paid) {
    await processPaymentEvent({
      provider: 'billplz',
      event_type: 'payment.success',
      amount: data.amount / 100, // Billplz sends in cents
      currency: 'MYR',
      member_id: data.reference_1,
      org_id: data.reference_2,
      reference: data.id,
      metadata: data,
```

```
    });
  }

  return c.json({ ok: true });
});
```

Malaysian Thermal Receipt Format

This was surprisingly specific. Malaysian gym owners who deal with older members (especially at morning sessions) often want to print physical receipts on thermal printers. The format expected is:

```
 ================================
         TIMEO GYM SYSTEM
================================
Resit No: TIM-2026-003847
Tarikh  : 19/03/2026 14:32
================================
Nama    : Ahmad Razif bin Rashid
No. IC  : 850312-14-XXXX
================================
BUTIRAN PEMBAYARAN:
Langganan Bulanan (Mar 2026)
              RM    99.00
--------------------------------
Subtotal              RM  99.00
SST (0%)              RM   0.00
JUMLAH                RM  99.00
================================
Kaedah Bayaran: FPX Maybank
================================
Terima kasih kerana memilih kami
     Dijana oleh Timeo v2.0
================================
```

(Note: SST on gym memberships is 0% in Malaysia as of writing, but the format expects the line to exist)

The bilingual format (Bahasa Melayu labels) was a conscious decision — it makes the receipt feel local and trustworthy to older Malay members.

```
export function generateThermalReceipt(payment: Payment, member: Member, org: Organizat
 const divider = '='.repeat(32);
 const thinDivider = '-'.repeat(32);
 const pad = (label: string, value: string, width = 32) => {
   const gap = width - label.length - value.length;
   return `${label}${' '.repeat(Math.max(0, gap))}${value}`;
 };

 return [
   divider,
   org.name.toUpperCase().padStart(16 + org.name.length / 2),
   divider,
   `Resit No: ${payment.receipt_number}`,
   `Tarikh  : ${formatMalaysianDate(payment.created_at)}`,
   divider,
   `Nama    : ${member.full_name}`,
   `No. IC  : ${maskIC(member.ic_number)}`,
   divider,
   'BUTIRAN PEMBAYARAN:',
   payment.description,
   pad('', `RM  ${formatMYR(payment.amount)}`),
   thinDivider,
   pad('Subtotal', `RM ${formatMYR(payment.amount)}`),
   pad(`SST (${payment.sst_rate}%)`, `RM ${formatMYR(payment.sst_amount)}`),
   pad('JUMLAH', `RM ${formatMYR(payment.total_amount)}`),
   divider,
   `Kaedah Bayaran: ${payment.payment_method_label}`,
   divider,
   'Terima kasih kerana memilih kami',
   `     Dijana oleh Timeo v${APP_VERSION}`,
   divider,
 ].join('\n');
}
```

E-Invoicing Compliance

Malaysia's MyInvois system (LHDN e-invoicing) requires businesses above RM 100M/year to submit digital invoices. While most of our gym clients were below that threshold, we built it in from Day 1 because clients grow and retrofitting compliance is a nightmare.

The LHDN API integration was the most complex piece of Day 3. I asked AI to generate the XML schema conforming to LHDN's technical specifications, then wrote the submission handler.

```
 // Simplified LHDN e-invoice structure
export async function submitEInvoice(invoice: Invoice): Promise<string> {
  const xmlDocument = buildLHDNXML({
    supplierTIN: process.env.LHDN_SUPPLIER_TIN!,
    invoice,
  });

  const signedDoc = await signWithDigitalCertificate(xmlDocument);

  const response = await fetch('https://myinvois.hasil.gov.my/api/v1.0/documentsubmissi
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${await getLHDNToken()}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      documents: [{
        format: 'XML',
        document: Buffer.from(signedDoc).toString('base64'),
        documentHash: sha256(signedDoc),
        codeNumber: invoice.invoice_number,
      }]
    })
  });

  const result = await response.json();
  return result.submissionUID;
}
```

Chapter 5: Day 4 — Hardware Integration (The Unexpected Rabbit Hole)

"Can It Open the Door?"

When I first heard this requirement from a potential client, I thought it was optional. Then I talked to five more gym owners and realized: hardware integration isn't a nice-to-have. It's the feature that justifies the entire platform switch.

"If your software can't talk to my turnstile, I'm not migrating. Simple."

Fair enough. Let's make it talk to turnstiles.

NFC Door Controllers

The most common NFC door controller in Malaysian gyms is the kind that speaks over RS-485 (a serial protocol) or, increasingly, over IP via a basic REST API or socket connection.

We partnered with a local hardware supplier and got access to their SDK. The protocol looked like this:

```
 // Door controller interface (hardware-agnostic)
export interface DoorController {
  ping(): Promise<boolean>;
  unlock(doorId: string, duration: number): Promise<void>;
  enrollCard(doorId: string, cardUid: string, memberId: string): Promise<void>;
  revokeCard(cardUid: string): Promise<void>;
  getLogs(since: Date): Promise<AccessLog[]>;
}

// IP-based controller implementation
export class IPDoorController implements DoorController {
  private baseUrl: string;
  private apiKey: string;

  constructor(config: { host: string; port: number; apiKey: string }) {
    this.baseUrl = `http://${config.host}:${config.port}`;
    this.apiKey = config.apiKey;
```

```
  }

  async unlock(doorId: string, durationMs = 5000): Promise<void> {
    await fetch(`${this.baseUrl}/doors/${doorId}/unlock`, {
      method: 'POST',
      headers: { 'X-Api-Key': this.apiKey },
      body: JSON.stringify({ duration: durationMs }),
    });
  }

  async enrollCard(doorId: string, cardUid: string, memberId: string): Promise<void> {
    await fetch(`${this.baseUrl}/cards/enroll`, {
      method: 'POST',
      headers: { 'X-Api-Key': this.apiKey },
      body: JSON.stringify({ cardUid, doorId, externalRef: memberId }),
    });
  }
}
```

The key design decision: make the door controller interface hardware-agnostic from day one. Different gyms use different hardware. Our `DoorController` interface means we can swap implementations without touching any business logic.

Facial Recognition Turnstile

This was the genuinely hard part. Facial recognition turnstiles operate differently from NFC — they need to be pre-loaded with face embeddings, and they need to either process recognition locally (on-device) or call back to our API.

The turnstile model we supported used a callback architecture:

```
 Turnstile sees face → Captures image → POSTs to our API →
We return { allow: true/false, member_id?, display_name? } →
Turnstile unlocks or displays error
```

Our face matching API:

```
app.post('/hardware/facial-recognition/verify', async (c) => {
  const { image_base64, terminal_id } = await c.req.json();

  // Get terminal config (which org, which branch)
  const terminal = await db.terminals.findFirst({
    where: { terminal_id },
    include: { branch: true }
  });

  if (!terminal) return c.json({ allow: false, reason: 'unknown_terminal' });

  // Convert base64 to buffer and extract face embedding
  const imageBuffer = Buffer.from(image_base64, 'base64');
  const embedding = await faceRecognition.extractEmbedding(imageBuffer);

  if (!embedding) return c.json({ allow: false, reason: 'no_face_detected' });

  // Search for matching member in this org (vector similarity search)
  const { data: match } = await supabase.rpc('find_matching_face', {
    p_org_id: terminal.branch.org_id,
    p_embedding: embedding,
    p_threshold: 0.85,
  });

  if (!match) return c.json({ allow: false, reason: 'no_match' });

  // Validate subscription
  const isValid = await validateMemberAccess(match.member_id);

  if (!isValid) {
    return c.json({ allow: false, reason: 'subscription_expired', display_name: match.fu
  }

  // Record check-in
  await recordCheckin(match.member_id, terminal.branch_id, 'facial');

  return c.json({ allow: true, member_id: match.member_id, display_name: match.full_name
});
```

The face embedding storage used pgvector (a PostgreSQL extension for vector similarity search). This was one of the more elegant technical decisions of the project — the same database that handles members and check-ins also handles the vector embeddings. No separate vector database.

```
 -- pgvector setup
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE member_face_embeddings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  member_id UUID REFERENCES members(id) ON DELETE CASCADE,
  org_id UUID NOT NULL,
  embedding vector(512),  -- FaceNet 512-dimension embeddings
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index for fast approximate nearest neighbor search
CREATE INDEX ON member_face_embeddings
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);

-- Function for similarity search within org
CREATE OR REPLACE FUNCTION find_matching_face(
  p_org_id UUID,
  p_embedding vector(512),
  p_threshold FLOAT
) RETURNS TABLE (member_id UUID, full_name TEXT, similarity FLOAT) AS $$
  SELECT m.id, m.full_name, 1 - (e.embedding <=> p_embedding) AS similarity
  FROM member_face_embeddings e
  JOIN members m ON e.member_id = m.id
  WHERE e.org_id = p_org_id
    AND m.status = 'active'
    AND 1 - (e.embedding <=> p_embedding) >= p_threshold
  ORDER BY similarity DESC
  LIMIT 1;
$$ LANGUAGE SQL;
```

The Real Lesson from Day 4

Hardware integration taught me something about AI-assisted development: AI is excellent at the "how to code this" parts. It's less good at the "what does this hardware actually do" parts.

I had to actually read the hardware SDKs, talk to the supplier, and test against real devices. AI generated the code patterns, but the domain knowledge had to come from me.

Rule of thumb: For any domain with proprietary hardware, budget 40% more time than AI estimates. The last 40% is bridging the gap between the ideal API and whatever the hardware actually does.

Chapter 6: Day 5 — Data Migration (The Night I Almost Quit)

381 Users. One Night. No Rollback Plan.

The gym we were migrating from had been running on the old system for 3 years. 381 active members. Their data was in a PostgreSQL database that the previous developer had set up, and which had been running fine until the client decided to switch.

The migration had to happen at 11 PM on a Wednesday — low traffic time for gyms — and be complete by 4 AM before the morning rush.

I almost quit at midnight.

What Made It Hard

The old schema was... creative. Some fields that should have been dates were stored as strings in DD/MM/YYYY format. Some IC numbers had spaces. Some members had duplicate entries from a bug in the old import tool. Member photos were stored as binary BLOBs in the database instead of files (why??).

Here's my migration analysis prompt:

```
I have two database schemas. I need to migrate all data from the old schema
to the new schema as part of a live production migration with a 5-hour window.
```

```
Old schema DDL:
[paste old schema]

New schema DDL:
[paste new schema]

Known issues I've discovered:
1. date_joined is stored as VARCHAR in DD/MM/YYYY format
2. Some ic_number entries have spaces (e.g., "850312 14 5671")
3. ~23 members have duplicate entries (same IC number, different IDs)
4. Photos are stored as BYTEA BLOBs in old DB
5. Old system uses INTEGER IDs; new system uses UUID

Please:
1. Write a migration script (TypeScript) that handles all these edge cases
2. Include a dry-run mode that shows what would be migrated without writing
3. Include a detailed log of every transformation applied
4. Include a verification step that runs post-migration and reports any discrepancies
5. Flag any records that need manual review instead of failing the whole migration

IMPORTANT: The script must be idempotent — safe to run multiple times.
```

The Migration Script

```
 import { createClient } from '@supabase/supabase-js';
import pg from 'pg';
import { v4 as uuidv4 } from 'uuid';

const OLD_DB = new pg.Client({ connectionString: process.env.OLD_DB_URL });
const newSupabase = createClient(process.env.SUPABASE_URL!, process.env.SUPABASE_SERVICE

interface MigrationReport {
  migrated: number;
  skipped: number;
  errors: Array<{ oldId: number; reason: string; data: unknown }>;
  manualReview: Array<{ oldId: number; reason: string }>;
}

async function migrateMembers(dryRun = true): Promise<MigrationReport> {
```

```
const report: MigrationReport = { migrated: 0, skipped: 0, errors: [], manualReview:

const { rows: oldMembers } = await OLD_DB.query(`
  SELECT * FROM gym_members
  ORDER BY id ASC
`);

console.log(`Processing ${oldMembers.length} members (dry run: ${dryRun})`);

// Build duplicate detection map
const seenIC = new Map<string, number>();

for (const old of oldMembers) {
  try {
    // Normalize IC number
    const normalizedIC = old.ic_number?.replace(/\s+/g, '').replace(/-/g, '') || null;

    // Detect duplicates
    if (normalizedIC && seenIC.has(normalizedIC)) {
      console.warn(`DUPLICATE: Member ${old.id} has same IC as ${seenIC.get(normalized
      report.manualReview.push({
        oldId: old.id,
        reason: `Duplicate IC number: ${normalizedIC}, first seen at old ID ${seenIC.g
      });
      continue;
    }
    if (normalizedIC) seenIC.set(normalizedIC, old.id);

    // Parse date_joined
    let joinedAt: Date;
    if (old.date_joined && old.date_joined.includes('/')) {
      const [day, month, year] = old.date_joined.split('/');
      joinedAt = new Date(`${year}-${month}-${day}`);
      if (isNaN(joinedAt.getTime())) throw new Error(`Invalid date: ${old.date_joined}
    } else {
      joinedAt = new Date(old.date_joined || Date.now());
    }

    // Extract photo from BLOB
    let photoUrl: string | null = null;
    if (old.photo_blob && !dryRun) {
```

```
        photoUrl = await uploadPhotoToR2(old.photo_blob, old.id);
      }

      const newMember = {
        id: uuidv4(),
        org_id: TARGET_ORG_ID,
        full_name: old.full_name?.trim() || 'Unknown',
        phone: normalizePhone(old.phone),
        email: old.email?.toLowerCase().trim() || null,
        ic_number: normalizedIC,
        photo_url: photoUrl,
        status: old.is_active ? 'active' : 'inactive',
        created_at: joinedAt.toISOString(),
        migrated_from_id: old.id, // Store original ID for reference
      };

      if (!dryRun) {
        const { error } = await newSupabase.from('members').upsert(newMember, {
          onConflict: 'migrated_from_id',
        });
        if (error) throw error;
      }

      console.log(`✅ [${old.id}] ${newMember.full_name} → ${newMember.id}`);
      report.migrated++;

    } catch (err) {
      console.error(`❌ [${old.id}] Error:`, err);
      report.errors.push({ oldId: old.id, reason: String(err), data: old });
    }
  }

  return report;
}
```

The Midnight Crisis

At midnight, the dry run had completed with 23 duplicate records flagged and 7 error records with invalid dates. I was tired. I wanted to skip the duplicates and just run it.

I didn't. I spent two hours on the phone with the client's gym manager, going through the 23 duplicates one by one, figuring out which record was the "real" one. We kept 22 and merged 1 (a guy who had genuinely signed up twice under different IC entries).

At 2:15 AM, the migration ran. 378 records migrated. 1 merged. 2 set to "pending review" status in the new system.

At 3:48 AM, the verification script ran: - Membership counts matched ✅ - Subscription statuses aligned ✅ - Active member access verified with QR scan test ✅ - Photos uploaded correctly ✅

At 4:00 AM, I went to bed.

Lesson learned: Data migrations are 10% coding and 90% data archaeology. Budget for the archaeology.

Chapter 7: Day 6 — Mobile Apps (Flutter and the App Store Gauntlet)

Why Flutter

I've built mobile apps in React Native, Expo, and native Swift/Kotlin. Flutter is my current favourite, and not for the reasons most people give.

The usual Flutter pitch is "one codebase, two platforms." That's true but table stakes in 2026.

The real reason I chose Flutter: **the widgets render the same everywhere**. In React Native, you're constantly fighting with platform-specific quirks. In Flutter, what you design is what ships. For a multi-tenant SaaS where I can't control the gym's phone model, that consistency is priceless.

The App Features (Day 6 Scope)

The Timeo mobile app needed to: 1. Allow members to view their membership status 2. Show their QR code for scanning at the gym 3. View check-in history 4. Pay for subscriptions or renewals 5. Allow gym staff to do manual check-ins (scan QR or manual lookup) 6. Offline mode for check-in scanning when internet is spotty

The staff functionality in particular was a Day 6 surprise — one gym owner called me at 7 AM and said "your web dashboard is great, but my staff are on their phones all day, not sitting at computers." Valid. We added it.

The Flutter Project Structure

```
mobile/
├── lib/
│   ├── main.dart
│   ├── app/
│   │   ├── router.dart        # GoRouter configuration
│   │   └── theme.dart         # Design tokens matching web
│   ├── features/
│   │   ├── auth/
│   │   ├── member/
│   │   │   ├── data/          # API calls
│   │   │   ├── domain/        # Business logic
│   │   │   └── presentation/  # Widgets
│   │   ├── checkin/
│   │   ├── qr_code/
│   │   └── payments/
│   └── shared/
│       ├── widgets/
│       ├── services/
│       └── utils/
```

Feature-first organization. Not layer-first. When you need to change anything about "check-in," you go to `features/checkin/` and everything you need is there.

The QR Scanner

```
// features/checkin/presentation/scan_checkin_page.dart
class ScanCheckinPage extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final checkinState = ref.watch(checkinProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Scan Check-In')),
```

```
      body: Stack(
        children: [
          MobileScanner(
            onDetect: (capture) {
              final barcode = capture.barcodes.firstOrNull;
              if (barcode?.rawValue != null) {
                ref.read(checkinProvider.notifier)
                   .processQRCheckin(barcode!.rawValue!);
              }
            },
          ),
          // Overlay showing last scan result
          if (checkinState.lastResult != null)
            CheckinResultOverlay(result: checkinState.lastResult!),
        ],
      ),
    );
  }
}

// The checkin notifier
class CheckinNotifier extends StateNotifier<CheckinState> {
  final CheckinRepository _repo;

  Future<void> processQRCheckin(String token) async {
    // Debounce rapid scans
    if (_lastScanTime != null &&
        DateTime.now().difference(_lastScanTime!) < const Duration(seconds: 2)) {
      return;
    }
    _lastScanTime = DateTime.now();

    state = state.copyWith(isProcessing: true);

    try {
      final result = await _repo.processCheckin(token);
      state = state.copyWith(
        isProcessing: false,
        lastResult: result,
      );
```

```
      // Haptic feedback
      if (result.success) {
        HapticFeedback.mediumImpact();
      } else {
        HapticFeedback.heavyImpact();
      }

    } catch (e) {
      state = state.copyWith(isProcessing: false, error: e.toString());
    }
  }
}
```

Offline Mode

Offline check-in was the feature that required the most architectural thought. When a gym's internet is down (more common in Malaysia's smaller cities than you'd think), staff can't tell members "sorry, can't let you in, WiFi is down."

Solution: **Offline-first QR validation**.

The device caches a daily snapshot of active member QR token hashes. When offline, it validates against the local cache and queues the check-in for sync when connectivity returns.

```
// Offline cache service
class OfflineCheckinCache {
  static const _cacheKey = 'active_member_tokens';

  Future<void> refreshCache(List<String> tokenHashes) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setStringList(_cacheKey, tokenHashes);
    await prefs.setInt('${_cacheKey}_refreshed', DateTime.now().millisecondsSinceEpoch);
  }

  Future<bool> validateToken(String tokenHash) async {
    final prefs = await SharedPreferences.getInstance();
    final cached = prefs.getStringList(_cacheKey) ?? [];
    return cached.contains(tokenHash);
  }
```

```
  Future<bool> isCacheStale() async {
    final prefs = await SharedPreferences.getInstance();
    final refreshed = prefs.getInt('${_cacheKey}_refreshed');
    if (refreshed == null) return true;
    final age = DateTime.now().millisecondsSinceEpoch - refreshed;
    return age > const Duration(hours: 12).inMilliseconds;
  }
}
```

Automated App Store Submission

This is where Fastlane earned its keep.

```
 # Fastfile
lane :deploy_ios do
  match(type: "appstore")
  build_app(
    scheme: "Timeo",
    export_method: "app-store",
    output_directory: "./build/ios"
  )
  upload_to_app_store(
    skip_metadata: false,
    skip_screenshots: true,
    submit_for_review: true,
    automatic_release: false,  # Manual release after review
    force: true
  )
end

lane :deploy_android do
  gradle(
    task: "bundle",
    build_type: "Release",
    project_dir: "android/"
  )
  upload_to_play_store(
    track: "internal",
    aab: "build/app/outputs/bundle/release/app-release.aab"
```

```
    )
end
```

The iOS submission was queued in under 10 minutes. The Play Store internal track was live within 30 minutes. App review took 3 days (Apple), 2 hours (Google). By end of Day 6, both submissions were in review.

Chapter 8: Day 7 — Polish (The Day Everything Becomes Real)

"It Works" ≠ "It's Ready"

There's a version of a product that works and a version that's ready. The gap between them is usually: - Ugly edge cases in the UI - Error messages that say "Internal Server Error" instead of something helpful - Flows that work for a 30-year-old tech-literate person but not a 65-year-old gym regular - A logo that looks like it was made in 2009

Day 7 was about closing that gap.

Logo Iterations

I went through 14 logo iterations with AI before landing on something I liked. The final process:

1. Describe the brand in words to Claude: *"Timeo is a gym management SaaS. Professional, modern, slightly bold. The name comes from a Latin concept of time. The product is Malaysian but should feel international. Think tech-forward but approachable."*
2. Generate prompt for image AI: `Minimalist logo for 'TIMEO', tech SaaS, geometric letterform, clock or time motif subtle, dark navy + electric blue, single icon + wordmark, SVG-friendly`
3. Iterate 10 times on variations
4. Take the best two into Figma for manual refinement

5. Export as SVG, optimize with SVGO, generate all required sizes

The final logo is a stylized T with a subtle arc suggesting both time and forward motion. Simple enough to read at 16px, distinctive enough to be memorable at 512px (the App Store icon size).

Elderly-Friendly UX

This requirement came from the client feedback and was genuinely important. Many gym members are 50-70+ years old. The original UI had 12px labels, gray placeholder text, and touch targets that would frustrate anyone over 45.

The changes: - Minimum touch target: 48×48px (Apple HIG standard, which is larger than most apps use) - Font size: minimum 16px for body text, 20px for important labels - High contrast: WCAG AA minimum for all text - Error states: clear, specific, non-technical language - Success confirmations: large, obvious, "Check-in confirmed!" with the member's name — not a subtle toast notification - Bilingual: Bahasa Malaysia option for members who prefer it

The AI prompt that generated most of the accessibility review:

```
 Review this UI component for accessibility issues, specifically:
- Touch target sizes (minimum 48×48px)
- Color contrast ratios (WCAG AA = 4.5:1 for normal text)
- Label clarity for non-technical users aged 50+
- Error message clarity
- Loading state feedback

[paste component code]

For each issue found:
1. Describe the problem
2. Provide the fix
3. Explain why it matters for elderly users specifically
```

QA Process

Day 7 QA was structured. Not ad-hoc clicking around. Each feature tested from three perspectives:

Persona 1: Gym Owner (Admin) - Log in as owner - Create a staff account - Verify RBAC limits what staff can see/do - Process a payment manually - Print a receipt - View revenue reports

Persona 2: Staff Member - Log in as staff - Process a QR check-in - Handle a failed check-in (expired subscription) - Add a new member - Cannot access billing settings (RBAC check)

Persona 3: Gym Member (Mobile) - Log in on iOS - View membership status - Find QR code - View check-in history - Renew subscription via FPX

Every bug found got a GitHub issue. Every issue got fixed before shipping. The QA checklist (full version in the Appendix) ran to 87 items.

The Deploy Sequence

```
 # 1. Run final test suite
pnpm test --all

# 2. Database migrations (Supabase)
supabase db push --linked

# 3. Deploy API workers
wrangler deploy --env production

# 4. Deploy frontend
vercel --prod

# 5. Health check all endpoints
./scripts/health-check.sh

# 6. Manual smoke test (the 3 personas above)
echo "Smoke test manually before announcing"
```

Chapter 9: The Agent Workflow — PLAN → REVIEW → BUILD → QA → SHIP

The Workflow That Made It Possible

Timeo wasn't built by Claude. It was built by me, with Claude as a force multiplier. The distinction matters.

The workflow I use is called PLAN → REVIEW → BUILD → QA → SHIP. I've run this for every feature, every migration, every major decision.

```
 PLAN    → Define what we're building and why
REVIEW  → Validate the plan with AI, find holes
BUILD   → AI-assisted implementation
QA      → AI-assisted testing and review
SHIP    → Deploy with verification
```

PLAN Phase

Before writing any code, I write a spec. Not a formal spec — a working document.

```
 ## Feature: Member QR Check-in

### What
Members scan a QR code at the gym entrance. System validates their active subscription
and records the check-in.

### Why
Core feature. Replaces manual sign-in sheets. Also enables automated door unlocking
(hardware integration, Day 4).

### Scope (this PR)
- QR token generation per member
- QR display page (web + mobile)
- Check-in validation endpoint
- Check-in record creation
- Success/failure response for scan UI

### Out of scope (future)
- NFC (separate implementation)
- Facial recognition (separate implementation)
- Offline mode (Day 6)
```

```
### Success criteria
- Member can scan QR and see "Welcome, [Name]!" in < 1 second
- Expired subscription shows clear "Subscription expired" message
- Every check-in recorded with timestamp and branch

### Risks
- QR token security (need to sign tokens, not use raw IDs)
- Race conditions if member scans twice quickly
```

REVIEW Phase

I paste the spec to Claude Opus with this preamble:

```
 You are acting as a senior engineer reviewing a feature spec before implementation.

Your job is to:
1. Identify any ambiguities in the spec
2. Flag technical risks I haven't considered
3. Suggest better approaches if my proposed design has issues
4. Ask clarifying questions about anything unclear

Be critical. I'd rather hear about problems now than discover them in production.
Don't build anything yet — just review.

[paste spec]
```

Opus usually comes back with 3-7 concerns. Some are obvious (I already thought of them). Some are things I missed. The missed ones are gold.

For the QR check-in feature, Opus flagged: - "What happens if a member's subscription expires mid-day? Do pending check-ins fail retroactively?" → No, check-in at time of scan, not retroactive. - "Should check-in be idempotent within a time window?" → Yes, added 2-minute debounce. - "How do you handle timezone for the branch — the check-in timestamp should use branch timezone, not UTC" → Good catch. Added branch timezone to check-in record.

BUILD Phase

After the review, I switch to Sonnet for the actual implementation. Why?

Opus is better at: - Architecture decisions - Spotting logical errors - Generating novel solutions to hard problems - Reviewing code for correctness

Sonnet is better at: - Fast, high-volume code generation - Iterating on implementations - Writing tests - Boilerplate-heavy tasks

Opus costs more. Sonnet is faster. Use them appropriately.

The build prompt follows the "specific implementation" template I showed in Chapter 3. Always include: schema context, existing types, error handling patterns, and "no TODOs."

QA Phase

After building, I run AI-assisted code review:

```
 Review this code for:
1. Security issues (injection, auth bypass, data leakage)
2. Missing error handling
3. N+1 database queries
4. Missing input validation
5. Edge cases not covered by the spec
6. TypeScript type safety issues

[paste code]

For each issue: severity (CRITICAL/HIGH/MEDIUM/LOW), description, fix.
```

Then I run the QA checklist (Appendix C) against the live feature.

SHIP Phase

Only after PLAN → REVIEW → BUILD → QA does code go to production. No exceptions. The workflow sounds slow but the opposite is true — it's faster because you catch bugs before they're in production, where they cost 10x more to fix.

The Subagent Pattern

For larger tasks, I spawned subagents — separate AI sessions with specific mandates. The subagent prompt template is in Appendix A.

Example subagent task:

```
 TASK: Implement the complete data migration script for Timeo.

Context:
- Old DB: [connection string for staging copy]
- New DB: Supabase project [project ID]
- Org ID for migration target: [org UUID]

Spec:
[paste full migration spec]

Rules:
- TypeScript only
- Idempotent (safe to run multiple times)
- Dry-run mode required
- Detailed logging to ./logs/migration-[timestamp].log
- Must produce a verification report post-migration

Output: Complete, tested, production-ready TypeScript script.
Do NOT use any packages not already in package.json.
```

The subagent does its work, returns the code, I review it. Parallel work, async review. This is how one person does the work of a team.

Chapter 10: The Numbers — What It Actually Cost

Development Costs

Let me be honest about the numbers.

AI API costs (7 days of heavy usage): - Claude Opus (architecture, review): ~$45 - Claude Sonnet (implementation, QA): ~$78 - Total AI: **~$123**

Infrastructure (first month): - Supabase Pro plan: $25 - Vercel Pro: $20 - Cloudflare Workers (paid): $5 - Cloudflare R2 storage: $3 - Resend (email): $20 - Billplz (payment gateway setup fee): RM 200 (~$45) - Total infra first month: **~$118**

Domain and miscellaneous: - Domain (timeo.my): RM 80 (~$18) - Apple Developer Account: $99/year (amortized: $8/month) - Google Play: $25 one-time (already had account) - Total misc: **~$51**

Total 7-day build cost: ~$292

Revenue Model

```
 Tier 1 (Starter):  RM 99/month  (~$22)  — up to 100 members, 1 branch
Tier 2 (Growth):   RM 199/month (~$44)  — up to 500 members, 3 branches
Tier 3 (Scale):    RM 399/month (~$88)  — unlimited, unlimited
Hardware bundle:   RM 1,500 one-time (~$330) — NFC controller + tablet
```

Month 1 Revenue (Real Numbers, Obfuscated for Privacy)

I'm not going to give you exact gym names or client data. But here's the shape of month 1:

- **Paying gyms at end of month 1:** 7
- **Plan distribution:** 4 × Starter, 2 × Growth, 1 × Scale
- **Hardware bundles sold:** 3
- **MRR at month 1:** RM 1,494 (~$330)
- **Hardware revenue:** RM 4,500 (~$990)
- **Month 1 total revenue:** RM 5,994 (~$1,320)
- **Month 1 costs:** ~$292 build + ~$118 infra = ~$410
- **Month 1 profit:** ~$910

Month 1 isn't the story. The story is the trajectory. With word-of-mouth in the Malaysian gym community (it's smaller than you think — gym owners know each other), we were at 23 paying gyms by month 3.

MRR at month 3: ~RM 4,200 (~$930) **Annualized run rate:** ~RM 50,400 (~$11,200)

Not quit-your-job money yet. But for a product built in 7 days, with $292 in development costs, the unit economics are excellent. The marginal cost of adding a new gym is nearly zero (infra scales, support doesn't scale proportionally).

The Real ROI

Here's the number people don't calculate: **opportunity cost of NOT shipping fast**.

Every week you spend planning instead of building is a week you're not getting feedback. Every week you're not getting feedback, you're building the wrong thing.

With a 7-day build, I got real customer feedback after one week. That feedback shaped the next month of development. The QR scanner debounce, the bilingual toggle, the staff mobile app, the thermal receipt format — all of these came from customer feedback in the first two weeks.

If I'd spent 3 months building "the perfect product," I would have built the wrong product.

Speed is a feature. Shipping is a strategy.

Appendix A: Sub-Agent Prompt Template

Use this template when delegating a bounded task to a sub-agent (another AI session, a coding agent, or even a developer).

```
 # Sub-Agent Task: [TASK_NAME]

## Context
You are working on [PRODUCT_NAME], a [brief description].

Tech stack: [list your stack]
Repository: [repo location or relevant path]

## Your Task
[Clear, specific description of what needs to be built/done]

## Scope
### In scope:
```

```
- [Specific thing 1]
- [Specific thing 2]
- [Specific thing 3]

### Out of scope (do NOT build):
- [Thing that sounds related but isn't needed]
- [Feature for future phase]

## Technical Requirements
- Language/Framework: [specify]
- Must follow existing patterns in: [file/directory]
- Use these existing utilities: [list]
- Do NOT introduce new dependencies without asking

## Input
[Paste relevant schema, types, existing code, or context]

## Expected Output
[Precise description of what the agent should produce]
- File paths and names
- Function signatures
- API shapes
- Test requirements

## Constraints
- No TODOs or placeholder implementations
- All code must be production-ready
- TypeScript strict mode (or equivalent)
- Error handling required for all async operations
- Log meaningful errors, not just catch and swallow

## Definition of Done
- [ ] [Specific testable criterion 1]
- [ ] [Specific testable criterion 2]
- [ ] [Specific testable criterion 3]
```

Appendix B: TIMEO_RULES.md Template

Every project I build gets a RULES.md file. This is the contract between me and every AI agent that touches the codebase. Customize for your project.

```
 # [PROJECT_NAME] RULES

## The Prime Directive
This codebase is maintained by AI agents and humans working together.
Every contribution must leave the codebase better than it found it.

## Architecture
- **Frontend:** Next.js 15 App Router + TypeScript (strict)
- **API:** Hono on Cloudflare Workers
- **Database:** PostgreSQL via Supabase (RLS enabled)
- **Mobile:** Flutter
- **Auth:** Supabase Auth (JWT)
- **Storage:** Cloudflare R2

## Code Standards

### TypeScript
- Strict mode always
- No `any` — use `unknown` and narrow it
- No non-null assertions (`!`) without a comment explaining why
- All async functions must handle errors explicitly

### API Design
- All responses: `{ data: T } | { error: string, code: string }`
- Use HTTP status codes correctly
- Validate all inputs with Zod
- Never trust client-supplied org_id — read from JWT

### Database
- Never use raw IDs in WHERE clauses without RLS context set
- All mutations must be wrapped in transactions if >1 table
- Never SELECT * — always specify columns
- Index any column used in WHERE clauses

### Security
- All endpoints require auth middleware unless explicitly public
- RBAC check must happen before any data access
- Never log PII (IC numbers, full names) in production
```

```
- Rate limit all public endpoints

## Multi-Tenancy Rules
- NEVER access data without setting `app.current_org_id`
- NEVER trust `org_id` from request body — always from JWT
- RLS is a safety net, NOT the primary isolation mechanism
- Test tenant isolation in QA for every new feature

## What We DON'T Do
- No vendor lock-in to single-cloud solutions
- No storing credentials in code or version control
- No breaking changes to mobile API without versioning
- No deploying on Fridays after 4 PM

## When In Doubt
1. Ask before breaking
2. Prefer reversible over irreversible
3. Prefer explicit over implicit
4. `trash` > `rm`
5. Write the test first, then the code

## Contact
- Technical questions: [maintainer]
- Business/product questions: [product owner]
- Emergency: [emergency contact]
```

Appendix C: QA Checklist

Run this before every production deploy. 87 items. I'm giving you the essential 40.

Authentication & Access Control

- [ ] Login works for all roles (owner, manager, staff, member)
- [ ] Logout clears session completely
- [ ] Expired tokens return 401, not 500
- [ ] Password reset flow completes end-to-end
- [ ] Staff cannot access billing settings

- [ ] Members cannot access staff dashboard
- [ ] Cross-tenant data access attempt returns 403
- [ ] Rate limiting triggers on repeated failed logins

Member Management

- [ ] Create member with all fields
- [ ] Create member with minimum required fields only
- [ ] Duplicate IC number is rejected with clear error
- [ ] Soft delete: member not visible in active list
- [ ] Soft delete: member's check-in history preserved
- [ ] QR code generates correctly for new member
- [ ] QR code is unique per member
- [ ] Member photo upload works (< 5MB, JPEG/PNG)

Check-In System

- [ ] Valid QR scan results in successful check-in
- [ ] Expired subscription QR returns clear message
- [ ] Invalid QR token returns error (not 500)
- [ ] Rapid double-scan debounce works (< 2s window)
- [ ] Check-in recorded with correct timestamp and branch
- [ ] Offline mode: cached tokens validate correctly
- [ ] Offline mode: check-ins sync on reconnect
- [ ] Hardware door unlock triggers on valid check-in

Payments & Receipts

- [ ] FPX payment flow completes end-to-end (staging)
- [ ] Card payment flow completes end-to-end (Stripe test mode)
- [ ] Failed payment does NOT activate subscription
- [ ] Receipt generates with correct member name and amount
- [ ] Receipt IC number is masked (last 4 digits only)
- [ ] Receipt MYR formatting correct (RM 99.00, not RM99 or MYR 99)
- [ ] SST line present even at 0%

- [ ] Thermal print output matches expected format

Mobile App

- [ ] App launches on iOS without crash
- [ ] App launches on Android without crash
- [ ] QR code displays clearly on phone screen
- [ ] Camera permission requested on first scan attempt
- [ ] Check-in history loads and paginates
- [ ] Subscription renewal payment completes on mobile
- [ ] Push notifications received (check-in confirmation)
- [ ] App works on slow 3G connection

General / Non-Functional

- [ ] All API errors return `{ error: string, code: string }` — no raw exceptions
- [ ] No console.log statements in production code
- [ ] No hardcoded credentials or IPs in codebase
- [ ] Database migrations run successfully on fresh database
- [ ] Health check endpoint returns 200 with status details
- [ ] All third-party API keys are in environment variables
- [ ] CORS configured correctly (only allowed origins)
- [ ] Rate limiting enabled on all public endpoints

Final Thoughts: What I'd Tell My Past Self

If I could go back and give myself one piece of advice before starting Timeo, it would be this:

The AI doesn't make you a better developer. The workflow does.

AI is a tool. Like a power drill versus a hand drill — same job, different speed. If you don't know how to drill a hole, a power drill doesn't help. But if you know what you're doing and you have a good workflow? You build a house in the time it used to take to build a room.

The workflow is PLAN → REVIEW → BUILD → QA → SHIP. The discipline is following it even when you're tired at 11 PM and want to skip the review step. The speed comes from the compound effect of making fewer mistakes, not from writing code faster.

Timeo works because we planned it well, built it on solid foundations, tested it rigorously, and shipped it before we overthought it.

Go build something.

Jabez | Oxloz LLC Kuala Lumpur, March 2026

If you found this useful, share it with another founder who needs it. If you have questions, reach out — I try to respond to everyone.

END OF EBOOK
